

USx Provider JSON — Project Guide

A handover guide for whoever runs this next. Written for someone who has never seen the project.

Generated 2026-09-09 · describes the repository at `USx_Provider_JSON` · 20 providers, 121 tools.

What this project does, in one paragraph. A police officer in a patrol car needs to look up a licence plate, a driver, a gun, or a boat in their state law-enforcement database. Mark43 software shows them a search form. *This repository holds the configuration files that define those forms and translate what the officer types into the exact message each state system expects.* One configuration per state system (a “provider”). Get it right and the officer gets an answer. Get it subtly wrong and the query silently returns nothing — or returns the wrong record.

Contents

1. The mental model
2. Repository map
3. The documents at the root
4. Anatomy of a provider
5. The two authorities (read twice)
6. The three commands
7. The tools, by job
8. What “done” means
9. How testing actually happens
- 9b. Deploying a JSON to a tenant — and proving it landed
10. Traps that have cost real time
11. Your first week
12. Glossary

1. The mental model

Four things happen between the officer and the state. This chain explains every file in the repository.

```
OFFICER types into a FORM
    |
    v
COMBINATION matching - the config decides WHICH query the typed fields amount to
    |
    v
WIRE XML             - the request actually sent to the state system
    |
    v
RESPONSE             - mapped back onto the officer screen
```

The form

Input boxes and dropdowns, defined in the provider JSON file. Called a **QIF** (Query Input Form). Each control has a `fieldId`, for example `LicensePlateNumber`.

The combination — the concept that matters most

A state system does not accept “whatever the officer typed”. It offers a fixed menu of specific queries, each requiring specific fields. A **combination** declares one of them:

Part	Meaning
set[]	Required. Every field listed must be filled or this query cannot run.
any[]	Optional. Sent if filled, ignored if not. Does not affect which query runs.
conditions[]	Gates. For example “only use this query if no VIN was entered”. This is how priority is enforced.
keyReference	An internal name for the combination. <i>Never sent to the state.</i> A label only.

The single most important rule in the project: the platform fires the **FIRST** combination whose requirements are met. Order is therefore not cosmetic — it is logic. If a loose combination sits above a specific one, it steals every search and the specific one can never run. Such a combination is called a **dead combo**, and several tools exist purely to find them.

2. Repository map

Path	What it is	Who edits it
providers/	The 20 deliverables. One folder per state system.	Build scripts, not you directly
tools/	121 PowerShell scripts: builders, validators, auditors, reporters.	You, carefully
tools/_*.ps1	18 shared modules (underscore prefix). Included by other scripts; never run directly.	You, very carefully — a change here touches all 20
tools/_probes/	8 one-off investigation scripts that proved worth keeping.	You
tools/config/	6 JSON lookup tables used by the tools.	Rarely
knowledge-base/	17 reference documents. The accumulated hard-won knowledge.	You, whenever you learn something
automation/extension/	A Chrome extension (10 files) that drives the real forms and captures evidence.	You
.claude/skills/	9 procedure documents written for the AI assistant, but plain enough for a human to use.	You

What is in the knowledge base

File	Use
README.txt	Index of the other sixteen. Start here.
BUILD_RULES.txt	The numbered rules a build must obey.
PLATFORM_CONSTRAINTS.txt	Anti-patterns and platform limitations — things that do not work and why.
QIDM_REFERENCE.txt	How combinations, requirements and conditions behave.
FIELD_REFERENCE.txt	Per-field conventions: names, state handling, dates, composite names.
TOOL_REFERENCE.txt	Why each tool exists , the incident that produced it, and what its PASS does and does not prove.
TESTING_REQUIREMENTS.txt	The testing rules and gates.
RULE_HANDLERS.txt , UNIVERSAL_SEARCH_HANDLERS.txt	The platform behaviours available for wiring fields. Check the second before concluding a capability does not exist.
IMPORT_ERRORS.txt	Import error text mapped to its cause.
others	Jira templates, production triage, per-provider constraints, a platform bug report.

3. The documents at the root

Read in this order. The first two are not optional.

File	Read it because
ENGINEERING_STANDARD.md	Read first. Defines what “done” means: the three laws and the six lifecycle stages.
CLAUDE.md	The operating manual — build rules, field conventions, tool index, workflow. Large; skim the headings and use it as a reference.
SESSION_STATE.md	Where things stand right now. Capped at 120 lines so it stays readable. Current state only, no history.
providers/IMPORT_LEDGER.md	Which version is installed on which customer system. Never answer “where is X installed?” from memory.
FINDINGS_REGISTER.md	Open problems, and — importantly — <i>rejected</i> fixes with the measurements that rejected them. Read before re-attempting anything.
REBUILD_TRACKER.md / REVERSE_PROPAGATION_LOG.md	Which providers still owe a pickup of a shared fix.
CHANGELOG.md / DEX_TICKETS.md	Release history and the Jira ticket map.
NEW_PROVIDER_PROCEDURE.txt	Step by step for adding a 21st provider.

The three laws (from ENGINEERING_STANDARD.md):

1. **The form comes first.** If the officer cannot use it, nothing else matters.
2. **A gate that cannot fail is not a gate.** A check that always passes proves nothing — which is why this project deliberately breaks its own configs to confirm the checks notice.
3. **Authority is directional, and both directions must be checked.** “Everything we built is allowed” and “everything required was built” are different questions.

4. Anatomy of a provider

All 20 folders share one shape. Using `providers/NJ_NJCJIS/` as the example:

Path	Meaning and use
<code>NJ_NJCJIS_v4.17.json</code>	THE DELIVERABLE. The file imported into the customer system. The version sits in the filename, so a version is never silently overwritten. <i>Exactly one JSON lives in the provider root.</i>
<code>source/*.pdf</code> and <code>_DEVDOC.txt</code>	The state developer manual. Authority on WHICH queries exist.
<code>source/*.xml</code>	The platform metadata. Authority on WHAT FIELDS each query needs.
<code>scripts/build_*.ps1</code>	The program that generates the JSON. You never hand-edit the JSON — you edit this and rebuild.
<code>docs/tracking/</code>	Hand-maintained, updated every version. Detailed below.
<code>docs/reports/</code>	16 auto-generated audit outputs. Disposable; regenerated on every build.
<code>docs/reference/</code>	Derived from the metadata: field definitions, supported queries, captured tenant dropdowns, test values.
<code>docs/deliverables/</code>	<code>OFFICER_GUIDE_*.html/.pdf</code> — the plain-language sheet a police department actually reads.
<code>logs/<Entity>/</code>	The evidence. One file per test holding the real request and response captured from a live system.
<code>logs/_archive_pre_v*/</code>	Evidence from superseded versions. A new version invalidates old evidence, so it is archived rather than deleted.

Inside `docs/tracking/`

File	Purpose
<code>*_BUILD_NOTES.txt</code>	The source of truth for history. What changed each version, and why.
<code>CHANGELOG_*.md</code>	Auto-generated from BUILD_NOTES. Do not hand-edit.
<code>*_STATUS.txt</code>	The test matrix and current state.
<code>*_SQVR.txt</code>	Supported Query Validation Report — every query path and whether it is confirmed working.
<code>*_ACCEPTED_DIVERGENCES.txt</code>	Reasoned exceptions. Where the build deliberately differs from an authority, with the evidence. This is what stops a known, accepted difference from re-appearing as a fresh problem forever.
<code>PENDING_UPDATES.txt</code>	Work that <i>blocks</i> this provider. Only lines WITHOUT a leading # block anything — commented lines are retired records kept for their reasoning.
<code>DEX_TICKET.md</code>	Pointer to the Jira ticket and the release notes posted there.
<code>JSON_INVENTORY.md</code>	Every version ever produced.
<code>BUILD_MANIFEST_*.json</code>	Hashes used by <code>enforce</code> to confirm the reports match the shipped file.

5. The two authorities — read this twice

Nearly every real defect this project has shipped came from misreading an authority, not from bad code. There are exactly two. They answer different questions, and confusing them is the classic mistake.

	The DEVDOC (state manual)	The METADATA (platform XML)
Answers	<i>Which queries may we build?</i>	<i>What fields does each query need?</i>
Authority over	Scope	Fields and requirements
Never use it for	Deciding whether a field is mandatory	Deciding whether a query is in scope

So: if the devdoc does not list a query as supported, do not build it — even if the metadata describes it. And if the metadata says a field is mandatory, it is mandatory — even if the state manual shows it in brackets as optional. Building to the manual in that case produces a request the system rejects.

The trap that has caused the most damage. The generated file `docs/reference/*_METADATA_REFERENCE.txt` is convenient but **flattens alternatives**. Where a query genuinely has three valid field combinations, that file shows one merged row. Building to the merged row produces a request that *no* alternative accepts. For any “is this field mandatory?” question, open the raw XML and read the `<Requirements>` of that specific combination. This is the one sanctioned use of the raw XML.

6. The three commands

The whole workflow reduces to three words, and each has exactly one command that owns it.

Word	Command	What happens
build	<code>tools\build_phase1.ps1 - Provider NAME</code>	Hands off. Proves the config against both authorities before a human is asked for anything: every documented query built, every optional actually transmitted, priority correct, requirements faithful, and the checks themselves verified able to fail.
test	<code>tools\test_phase2.ps1 - Provider NAME</code> then the same with <code>- PostIngest</code>	Runs the real forms on a real system through the browser extension, captures the actual requests, then verifies the captured evidence.
finalize	<code>tools\enforce.ps1 - Provider NAME</code>	The final gate. Exit code 0 is the definition of done. Then commit, push, and confirm the Jira entry and install record exist.

Two conveniences: `tools\pipeline.ps1 -Provider NAME` chains a build with all its reports and audits; `tools\doctor.ps1` is a read-only health dashboard for the whole repository.

If `enforce.ps1` exits 0, the work is done. If it does not, fix what it names. There is no separate checklist to

remember — removing that need is the entire point of the design.

7. The tools, by job

121 scripts is a lot. You do not need them individually — you need to know which *job* each group does. The reasoning behind every one is in `knowledge-base/TOOL_REFERENCE.txt`.

Job	Representative tools
Build the config	<code>providers/*/scripts/build_*.ps1</code> , with shared modules <code>_build_layout_helpers</code> , <code>_build_rms_bundle</code> , <code>_build_provider_helpers</code>
Check structure	<code>validate.ps1</code> (six-phase structural validator), <code>verify_build.ps1</code>
Check against the authorities	<code>audit_metadata</code> , <code>audit_devdoc_combinations</code> , <code>audit_devdoc_optionals</code> , <code>audit_requirement_fidelity</code> , <code>audit_supported_queries</code>
Check the logic	<code>audit_combo_reachability</code> (dead combos), <code>audit_prefill_shadow</code> , <code>audit_devdoc_order</code> , <code>audit_wiring_closure</code> (does every control actually reach the wire?)
Check the evidence	<code>audit_log_content</code> , <code>audit_log_combo_attribution</code> , <code>audit_log_inflation</code> (are N logs really N distinct tests?), <code>audit_artifact_provenance</code>
Check the checks	<code>audit_gate_efficacy</code> (breaks the config on purpose to confirm each check notices), <code>fuzz_gate_efficacy</code> (random breakage, aimed at nothing)
Report	<code>portfolio_status</code> (the canonical one-screen table), <code>report_mission_status</code> , <code>report_import_owed</code> , <code>render_officer_guide</code>
Orchestrate	<code>build_phase1</code> , <code>test_phase2</code> , <code>enforce</code> , <code>pipeline</code> , <code>doctor</code>

Why “check the checks” exists, and why it is not paranoia. A check that always passes and a config that is genuinely correct look *identical* from outside. So `audit_gate_efficacy` injects each known kind of defect into a throwaway copy and confirms the responsible check fails. If it does not, that check’s green light meant nothing. The current figure is **261 injected defects, 0 undetected**. This has repeatedly found checks that were silently blind — including, on the day this guide was written, a blocking check that could not see the simplest kind of dead combination.

8. What “done” means

Six stages. A provider is finished only when all six hold. `tools\report_mission_status.ps1` computes this rather than anyone tracking it by hand.

#	Stage	Satisfied when
1	Build	Structural validator reports 0 failures and 0 warnings
2	Spec	Every query the devdoc lists is built
3	Reachability	Every built combination can actually run
4	Test	All five entities pass on a real system, with captured evidence
5	Jira	The ticket names the shipped version
6	Import	The ledger accounts for that version — installed, or explicitly not yet

At the time of writing: 13 of 20 complete. Five need a tenant test, one needs a Jira comment, one is deliberately parked.

9. How testing actually happens

Testing is not simulated. A real query goes to a real state system and the real request is captured.

1. `tools\emit_test_plan.ps1` produces a machine-readable plan: every combination, with values to type.
2. A **Chrome extension** is loaded in the browser. It fills the form and clicks Send, then scrapes the request XML from the platform own log page.
3. `tools\watch_captures.ps1` watches the Downloads folder and ingests each captured file.
4. Four checks verify the evidence: content matches the plan, the combination that fired is the one claimed, and the logs are genuinely distinct tests rather than duplicates.

Extension file	Role
<code>manifest.json</code>	Declares which pages the extension may touch. This allowlist is a safety control — it is what stops the driver running on a live customer site.
<code>usx_lib.js</code>	Shared primitives: fill a field, pick from a dropdown, click Send, extract the request XML.
<code>driver.js</code>	Runs a plan — one combination or a whole entity.
<code>capture.js</code>	Scrapes the request and response from the platform log page.
<code>ui.js</code>	The on-screen panel, so the work is buttons rather than typed commands.
<code>authwatch.js</code>	Diagnostic added 2026-09-09 for a ticket about a missing warning icon.
<code>bridge.js</code> , <code>background.js</code> , <code>nethook.js</code>	Plumbing: downloads and network interception.

Two safety rules. The extension is **disarmed by default** on every site and must be armed with an on-screen switch, because some tenants are live systems where real officers run real queries. And **no browser dialogs are ever used**: Chrome lets a user suppress them, and a suppressed confirmation silently returns “no” — which once made the arming switch itself unusable.

A rebuild invalidates all testing. Changing the config means the old evidence describes a file that no longer ships. Logs are archived automatically and the test matrix restarts. This is deliberate: re-test cost is never a reason to skip a fix.

9b. Deploying a JSON to a tenant — and proving it landed

Everything above gets a configuration *correct*. This section is how it gets *installed*, and how you know it actually did. Built 2026-09-11/12; before that, imports were done by hand and recorded from memory.

The decision is a file, not a button press. The operator used to assemble the decision at the keyboard — open a tenant, read a panel, judge a dry run, press a red button. Decision and execution were one act, so there was nothing to review beforehand and nothing to diff afterwards. Now `emit_import_job.ps1` writes `providers\IMPORT_JOB.json`, a human reads it, and the panel button executes only what it says.

The loop, end to end:

1. `tools\emit_import_job.ps1 -Provider FL_FCIC (or -DeptId, or -All)` — queues a tenant only when a bundle **hash** differs from the repo build. Never on a label.
2. **Read the job.** It states from-version, to-version, the exact bundles that will change, tenant status, and whether an override was used.
3. Open the target URL → USx panel → DEPLOY → **RUN THE JOB FOR THIS TENANT**. No provider is typed: it comes from the repo record via `GET /target/<deptId>`, and the payload from `GET /build/<PROVIDER>` byte-for-byte.
4. The panel re-exports the tenant automatically once the import is clicked.
5. `tools\watch_imports.ps1 -Once` — ingests that export and runs `verify_tenant_import.ps1`: **PASS / THE IMPORT DID NOT LAND / LABEL-ONLY CHANGE / UNPROVEN / FAIL**.
6. `tools\propose_ledger_patch.ps1` — proposes ledger corrections with line numbers and evidence. It **never writes** `IMPORT_LEDGER.md`.

An import REPLACES the bundle set. It does not merge. Proven on `usx-fl-fcic`: the tenant's `CA_eSUN` bundle was *removed* by an `FL_FCIC` import, not left alongside. A partial payload therefore deletes whatever it omits — which is why the guards insist on a complete, parseable, version-stamped build with ENTITIES and exactly one provider bundle.

Two checks that look right and are wrong. A **label check passes on a no-op** — the version rides in a bundle *description*, and four tenants carry stale descriptions over current content. A **raw byte compare fails on a correct import** — the platform re-serializes on export, emitting `"conditions":null` where our build omits the property. Both are avoided by `tools\_bundle_identity.ps1`; any new tool that compares tenant content *must* use it.

Only one file can write. `automation/extension/deploy_probe.js` — separate from the read-only `admin_probe.js` so “is the read path still read-only?” is answerable by reading one file. Dry-run by default; `execute:true` required; one tenant per call. Its guards are mutation-tested by `tools\audit_deploy_guard.ps1` (46 cases, including a valid CONTROL that must be *allowed* — a guard set that refused everything would score perfectly and block every deploy).

GUI only. Never a console command. The job runner shipped briefly as `__usxJob()` and was corrected within the hour. A mechanism becomes a *control*, never an instruction to type.

Scoping and grouping. `tools\config\tenant_scope.json` decides which tenants are in scope, with a reason and a date per exclusion. It ships **empty** and nothing is excluded until a human says so.

`tools\report_tenant_groups.ps1` renders every discovered tenant as a PDF grouped ours / not-ours / real-looking / development, with the signal that placed each row, so the decision is made by looking rather than by trusting a heuristic.

Where the versions come from. Never from the admin table's `Version` column — that is a *platform counter*, not ours. `audit_tenant_provenance.ps1` identifies a bundle by content against every build in git history;

`audit_tenant.ps1` gives the one-tenant dossier and prints label and content side by side, never collapsed into a single “tenant version”.

10. Traps that have cost real time

Trap	What to do instead
Prefilling a field that decides routing. Giving a form field a default makes it always present, which permanently hides every query that required its absence. This silently killed 35 query paths across 6 providers.	Never set a default on a field appearing in any <code>set[]</code> . Put it in the combination <code>defaults[]</code> instead, which does not affect routing.
Treating another provider as precedent. Two providers can look identical and have opposite correct answers, because their state manuals differ.	Read <i>that</i> provider own authority every time. Five conclusions were retracted in a single day for this.
Trusting a stale note. Notes saying work is owed repeatedly outlive the work, manufacturing phantom tasks.	Measure the current state with the tool before acting on any written claim — including one in this guide.
Reading a report without checking it is fresh. A leftover file from an earlier run satisfies “does the file exist?”.	Compare write times, never mere existence.
Counting commented-out lines as live. Retired records look identical to active flags.	In <code>PENDING_UPDATES.txt</code> , only lines without a leading <code>#</code> block anything.
PowerShell: <code>@(\$null).Count</code> is 1, not 0. This has produced wrong measurements <i>and</i> a real defect in a shipping check.	Test “property absent”, “is null” and “count” separately.
PowerShell 5.1 versus 7. The tools are run by 5.1; interactive work often uses 7. Grammar and error handling differ, so a script can be “verified” under 7 and be broken under 5.1.	Verify with <code>powershell</code> , not <code>pwsh</code> . <code>tools\audit_ps51_parse.ps1</code> checks all 129 scripts.

11. Your first week

1. **Day 1 — read.** `ENGINEERING_STANDARD.md`, then `SESSION_STATE.md`, then skim the headings of `CLAUDE.md`. Change nothing.
2. **Day 1 — look.** Run `tools\portfolio_status.ps1` and `tools\doctor.ps1`. Both are read-only. This is the current truth of the repository.
3. **Day 2 — one provider end to end.** Take `NJ_NJCJIS`, the reference implementation. Open its build script, its devdoc, its metadata XML and one test log. Trace one query from form field to wire XML.
4. **Day 3 — run the gates.** `tools\build_phase1.ps1 -Provider NJ_NJCJIS`, then `tools\enforce.ps1 -Provider NJ_NJCJIS`. Read what they print. Change nothing.
5. **Day 4 — break something on purpose.** `tools\audit_gate_efficacy.ps1 -Provider NJ_NJCJIS`. Watch it inject defects and confirm the checks catch them. This teaches the failure modes faster than reading about them.
6. **Day 5 — make one small change.** A field label. Rebuild with `pipeline.ps1`, watch every document update, and see enforce go green. Then you understand the loop.

The habit that matters most: run the tool that owns the question instead of reasoning about the answer. `CLAUDE.md` contains a routing table ("Source Authority Lookup Table") naming the one correct source for each kind of question. Nearly every mistake in this project history is someone answering from memory when a tool would have answered from fact.

12. Glossary

Term	Meaning
USx	Universal Search — the Mark43 feature this configures.
Provider	One state or regional law-enforcement system (TX_TLETS is Texas).
QIF	Query Input Form — the on-screen form.
QIDM	Query Input Data Mapping — the rules turning form input into a request. Holds the combinations.
Combination	One specific query with its required and optional fields.
keyRef	Internal label for a combination. Never transmitted.
Devdoc	The state developer manual. Authority on which queries exist.
Metadata	Platform XML. Authority on what fields a query requires.
Entity	One of the five search subjects: Person, Vehicle, Firearm, Article, Boat.
Dead combo	A combination that can never run because an earlier one always matches first.
Guardrail	A condition enforcing priority, such as preferring a plate over a VIN.
Gate	An automated check that can block the work.
Mutation testing	Deliberately breaking a copy of the config to prove the checks notice.
Tenant	One customer installation. Test tenants are safe; Foundation is staging; LIVE has real officers.
Wire XML	The actual request sent to the state system. The thing that ultimately matters.

Generated from the repository itself rather than from recollection: file counts, provider count, tool counts and status figures were measured at generation time. If a number here disagrees with `tools\portfolio_status.ps1`, the tool is right and this document is stale.